

Basic Abstractions in Distributed Algorithms III

Based on: Cachin, Guerraoui, Rodrigues

Danilo Kačanski

Faculty of Technical Sciences, Novi Sad
Applied algorithms in Control Systems

April 2026.

Runtime

What is the Runtime?

- The runtime is the **central scheduler** of the simulator.
- It combines all previous abstractions such as: processes, messages, links, failure models and cryptography.
- It is responsible for producing an actual **execution**.

What does it do?

- stores in-transit messages in a queue
- chooses which message is processed next
- applies failures and link logic
- delivers messages to processes
- records everything in a trace

Key Idea

The runtime is the **environment** in which distributed processes execute.

Why is the Runtime Single-Threaded?

- This simulator uses:
 - no goroutines
 - no channels
 - no real networking
- All nondeterminism comes from:
 - random message scheduling
 - explicit failure injection
- The random seed makes executions **reproducible**
 - same seed $\Rightarrow$ same execution

Why is this useful?

- easier debugging
- easier reasoning
- easier understanding abstractions for the first time

Execution Trace: Slightly Theoretical View

- In distributed systems, an **execution** is modeled as a sequence of events called **trace**:

$$e_1, e_2, e_3, \dots$$

- Each event represents an observable action in the system such as: sending a message, delivering a message, changing local state, crashing, recovering ...

Key Idea

We do not reason about *code running in parallel*, but about **all possible traces of events**.

Why is this important?

- Properties are defined over traces:
 - Safety: something bad never appears in the trace
 - Liveness: something good eventually appears in the trace

Execution Trace (runtime/trace.go)

- The trace records the **entire observable execution**.
- Every important action becomes an **event**.
- This allows:
 - debugging
 - later analysis
 - property checking

Event examples

```
SEND, ENQUEUE, DEQUEUE, DELIVER  
DROP, RETRANSMIT, CRASH, FORGE  
VIOLATION
```

Key Idea

The trace is the runtime's **memory of execution**.

Trace Event Structure

Event structure

```
type Event struct {  
    Step      int  
    Type      EventType  
    Message   *process.Message  
    Process   process.ProcessID  
    Detail    string  
}
```

Why is this useful?

Events are structured enough for checkers, but readable enough for humans.

Key Operations

- `EventLabel(t)`: returns a readable label for an event type
- `String()`: formats one event for display in the trace
- `Log(e)`: records an event and optionally prints it
- `Events()`: returns all recorded events
- `EventsOfType(eventType)`: filters events by category
- `Summary()`: prints aggregated statistics of the execution

Runtime Structure (runtime/runtime.go)

Core fields

```
type Runtime struct {  
    processes map[process.ProcessID]process.Process // maps ID to  
        process implementation  
    queue      []process.Message // in-transit messages  
    link       link.Link // communication abstraction  
    failures   failures.FailureInjector // failure model  
    trace      *Trace // execution log  
    checkers   []Checker // property of execution  
    rng        *rand.Rand  
    stepCount  int  
    maxSteps   int  
}
```

How to think about it

- queue = network
- link = communication guarantees
- failures = fault model
- runtime = scheduler that ties everything together

Key Idea

The runtime is a **controlled world** for experimenting with "distributed" executions.

Runtime Configuration

Basic setup methods

```
func (r *Runtime) Register(p process.Process)
func (r *Runtime) AddChecker(c Checker)
func (r *Runtime) SetInterceptor(
    fn func(process.Message) process.Message)
func (r *Runtime) SetVerbose(v bool)
```

- Register: add a process to the simulation
- AddChecker: add a property checker
- SetInterceptor: simulate a network attacker
- SetVerbose: control trace printing

Key Idea

Before execution starts, the runtime must be **configured**.

Message Injection

Injecting initial/system messages

```
func (r *Runtime) Inject(msg process.Message)
```

- Inject places a message directly into the queue
- It bypasses the link layer completely
- Useful for:
 - initial INIT messages
 - testing custom scenarios
 - system-generated events

Important

Injected messages are marked in metadata so the runtime knows they did not come through normal link processing.

One Simulation Step

- The runtime models execution as a sequence of **steps**.
- In each step, exactly one message is selected and processed.
- This gives a clean model of how asynchronous distributed execution would work.

High-level step structure

- ① process retransmissions
- ② choose one queued message
- ③ apply failures and link checks
- ④ deliver to process
- ⑤ enqueue generated outgoing messages

Key Idea

A step is the runtime's version of one **atomic transition** in the system.

Step() I: Scheduling

- ➊ **Increment step counter**
- ➋ **Call `link.Tick()`**
 - allows retransmissions (e.g. stubborn links)
- ➌ **Check if queue is empty**
 - if empty, no more execution is possible
- ➍ **Pick one random message from the queue**
 - models nondeterministic message delivery order

Why random dequeue?

In real systems, message order is determined by network timing and system behavior, which we cannot control. The simulator uses randomness to represent all possible valid executions.

Step() II: Filtering Before Delivery

- 5 **Apply interceptor**
 - simulates a network attacker modifying messages
- 6 **Check IsAlive**
 - crashed processes do not receive messages
- 7 **Check ShouldDeliver**
 - omission failures may drop the message
- 8 **Call `link.Receive`**
 - deduplication
 - authentication checks

Key Idea

Not every queued message will actually be delivered.

Step() III: Process Execution

- 9 Find recipient process
- 10 Log delivery in the trace
- 11 Call `Handle(msg)`
 - process performs one atomic step
 - may update local state
 - returns zero or more outgoing messages

Important

Processes do not send messages directly. They only **return outgoing messages** to the runtime.

Key Idea

The runtime controls communication; the process only defines reaction logic.

Step() IV: Outgoing Messages and Checking

- 12 **Apply MaybeAlter**
 - Byzantine failures may modify outgoing messages
- 13 **Call `link.Send` for each outgoing message**
 - link may drop, duplicate, authenticate, retransmit
- 14 **Enqueue resulting messages**
- 15 **Run safety checkers**
 - check bad things do not happen during execution

Key Idea

After one process reacts, the runtime turns its outputs into future network events.

Run(): Full Execution

Execution loop

```
func (r *Runtime) Run() {  
    for r.stepCount < r.maxSteps {  
        if !r.Step() {  
            break  
        }  
    }  
    // run liveness checks  
    // print trace summary  
}
```

- Run() repeatedly calls Step()
- Execution stops when:
 - the queue becomes empty, or
 - the maximum number of steps is reached
- At the end, liveness properties are checked

Safety vs Liveness in the Runtime

Safety

- “Something bad never happens”
- checked **during execution**, after each step
- example: no duplication

Liveness

- “Something good eventually happens”
- checked **at the end**
- example: reliable delivery

Key Idea

The runtime separates these two kinds of properties and checks them at the correct time.

Checkers (runtime/checker.go)

Checker structure

```
type Checker struct {  
    Name    string  
    Check  func(events []Event) string  
    AtEnd  bool  
}
```

- Name: property name
- Check: function that inspects trace events
- AtEnd: determines safety vs liveness

Examples

- NoCreationChecker()
- NoDuplicationChecker()
- ReliableDeliveryChecker()

How NoCreationChecker() Works

Goal

Verify the **no-creation** safety property: a message must never be delivered unless it was previously sent.

Method

- Maintain a set:
 - `sent[message] = true`
- Scan the trace from beginning to end
- For each SEND event:
 - record the message in the sent set
- For each DELIVER event:
 - check whether that message is already in sent
 - if not, report a violation immediately

How NoDuplicationChecker() Works

Goal

Verify the **no-duplication** safety property: the same message must not be delivered more than once.

Method

- Maintain a counter map:
 - `delivered[message]++`
- Scan the trace from beginning to end
- For each DELIVER event:
 - increment the counter for that message
 - if the count becomes greater than 1, report a violation

How ReliableDeliveryChecker() Works

Goal

Verify the **reliable-delivery** liveness property: every sent message should eventually be delivered.

Method

- Maintain two sets:
 - `sent[message] = true`
 - `delivered[message] = true`
- Scan the full trace
- Record every SEND event in `sent`
- Record every DELIVER event in `delivered`
- At the end, compare the two sets:
 - if some sent message is missing from `delivered`, report a concern